TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant (LangGraph + HITL)

1. HIGH LEVEL DESIGN (HLD)

1.1 Introduction

The RAG-Based Customer Support Assistant is an AI-powered system designed to answer user queries using a **Retrieval-Augmented Generation (RAG)** approach. It combines document retrieval, semantic search, and structured workflow execution to provide accurate and context-aware responses.

The system is enhanced with:

- LangGraph-style workflow orchestration
- Vector database for semantic search
- Human-in-the-Loop (HITL) escalation mechanism

1.2 Problem Statement

Traditional chatbots:

- Fail to understand context
- Give generic or hallucinated answers
- Lack grounding in real documents

This system solves those problems by:

- Grounding responses in PDF knowledge base
- Using embeddings for semantic retrieval
- Adding workflow control for structured execution
- Escalating uncertain queries to humans

1.3 SYSTEM ARCHITECTURE

The system architecture of the RAG-Based Customer Support Assistant is designed to simulate an end-to-end intelligent question answering system that combines document retrieval, semantic search, and structured decision-making.

The architecture follows a layered and modular design, where each layer performs a specific function in the overall workflow. This ensures scalability, maintainability, and clear separation of concerns.

Architectural Flow

The system operates in a sequential pipeline starting from user input and ending with either an AI-generated response or human escalation.

1. The user submits a query through a simple interface (CLI or chatbot).
2. The query is processed and classified into an intent category such as refund, shipping, or cancellation.
3. The query is converted into a vector representation using a pre-trained embedding model.
4. A vector database is searched to retrieve semantically similar document chunks.
5. The retrieved context is passed to a response generation module.
6. A final answer is generated strictly based on the retrieved information.
7. If confidence is low or context is insufficient, the query is escalated to a human agent.

Layer-wise Architecture Explanation

1. User Interface Layer

This layer handles interaction with the user. It accepts natural language queries and displays responses.

2. Intent Classification Layer

This layer identifies the purpose of the query (e.g., refund, shipping, cancellation). It helps route the query to the appropriate processing logic.

3. Embedding Layer

The input query and document text are converted into dense vector representations using a transformer-based embedding model. This allows semantic understanding rather than keyword matching.

4. Vector Storage Layer

All document embeddings are stored in a vector database such as ChromaDB. This enables efficient similarity search over large datasets.

5. Retrieval Layer

This layer performs semantic search by comparing query embeddings with stored document embeddings. It retrieves the most relevant text chunks based on similarity scores.

6. Response Generation Layer

The retrieved context is used to construct a final response. The system ensures that answers are grounded only in the provided context to avoid hallucination.

7. Workflow Orchestration Layer (LangGraph Concept)

This layer controls the flow of execution between different modules using a graph-based structure. Each node represents a processing step such as input, retrieval, or output generation.

8. Human-in-the-Loop (HITL) Layer

If the system is unable to confidently answer a query, it triggers escalation to a human agent. This ensures reliability and correctness in critical scenarios.

1.4 System Components

1. Document Loader

- Loads PDF documents using PyPDFLoader
- Extracts raw text from file

2. Chunking Module

- Splits large text into smaller chunks
- Uses RecursiveCharacterTextSplitter
- Improves retrieval accuracy

3. Embedding Module

- Model: sentence-transformers/all-MiniLM-L6-v2
- Converts text into dense vector representations
- Captures semantic meaning

4. Vector Database (ChromaDB)

- Stores embeddings efficiently
- Enables fast similarity search
- Supports persistence across sessions

5. Retrieval Module

- Performs cosine similarity search
- Retrieves top-K relevant chunks
- Filters most relevant context

6. Response Generation Module

- Extracts best matching sentences
- Uses retrieved context only
- Prevents hallucination
- Produces concise answers

7. Workflow Engine (LangGraph Simulation)

- Controls system execution flow
- Ensures structured node-based execution:
    - Input → Intent → Retrieval → Generation → Output

8. HITL (Human-in-the-Loop) Module

- Detects uncertain queries
- Escalates to human agent
- Ensures correctness in critical cases

1.5 Data Flow

1. User enters query
2. Query is classified (intent detection)
3. Query converted into embedding
4. Vector DB retrieves similar chunks
5. Context passed to response generator
6. Final answer is produced
7. If low confidence → HITL triggered

1.6 Technology Stack

- Python
- LangChain
- Sentence Transformers (MiniLM)
- ChromaDB / FAISS
- PyPDFLoader
- Custom workflow logic (LangGraph simulation)

1.7 Scalability

- Works with large PDFs using chunking
- Vector DB allows fast search at scale
- Modular architecture allows independent upgrades
- HITL ensures safe fallback for complex queries

2. LOW LEVEL DESIGN (LLD)

2.1 Module-Level Design

1. Ingestion Module

Responsibilities:

- Load PDF file
- Extract text
- Send data for chunking

2. Chunking Module

Responsibilities:

- Split text into chunks (300–500 tokens)
- Maintain semantic continuity
- Prepare input for embedding

3. Embedding Module

Responsibilities:

- Convert chunks into vectors
- Use transformer model
- Output numeric embeddings

4. Vector Storage Module

Responsibilities:

- Store embeddings in ChromaDB
- Enable persistence and retrieval

5. Retrieval Module

Responsibilities:

- Accept query embedding
- Perform similarity search
- Return top-K chunks

6. Query Processing Module

Responsibilities:

- Clean user input
- Detect intent (refund, cancel, shipping, etc.)

7. Response Generation Module

Responsibilities:

- Extract best matching sentence
- Build final response
- Ensure grounding in context

8. Graph Execution Module

Responsibilities:

- Simulate LangGraph workflow
- Control execution sequence
- Maintain state transitions

9. HITL Module

Responsibilities:

- Detect low confidence queries
- Escalate to human agent
- Handle fallback responses

2.2 Data Structures

Document Object

```
{
  "text": "string",
  "metadata": {}
}
```

Chunk Object

```
{
  "chunk_id": int,
  "text": str,
  "embedding": list
}
```

Query Object

```
{
  "query": str,
  "intent": str
}
```

State Object (LangGraph Simulation)

```
{
  "query": str,
  "intent": str,
  "chunks": list,
  "answer": str,
  "score": float,
  "escalate": bool
}
```

2.3 Workflow Design (LangGraph Style)

Nodes:

1. INPUT NODE
2. INTENT CLASSIFICATION NODE
3. RETRIEVAL NODE
4. GENERATION NODE
5. OUTPUT NODE
6. HITL NODE (optional)

2.4 HITL Design

Trigger Conditions:

- Low confidence retrieval
- Missing context
- Explicit human request

Process:

1. Query flagged
2. System pauses automation
3. Human provides response
4. Response returned to user

2.5 Interface Design

Input Format:

User Query (string)

Output Format:

```
{
  "answer": "final response",
  "intent": "classified intent",
  "sources": []
}
```

2.6 Error Handling

- PDF read failure → graceful exit
- Empty retrieval → fallback message
- Invalid query → general intent
- Vector DB failure → error recovery
- No context → escalation trigger